Report

Of

Basic Authentication Simulation

(AICTE – EduSkills Virtual Internship: Java Full Stack With Project)

Submitted

To

School of Computer Science and Engineering

IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement of degree of

B.Tech CSE



By

Roll Number of Student: CS-23411560

Name of Student: NIKITA PATLE

Batch: B.Tech CSE with IOS & 7th Semester

2026-27

CANDIDATE'S DECLARATION

I, Nikita Patle do hereby declare that the internship report titled **"Basic Authentication Simulation"** has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in CSE. All the references have been quoted and I have not taken as such any material from any other source. I affirm to you that this report is my own and has not been submitted to any other institute for any degree/diploma requirement and further I shall be solely responsible for any kind of copyright violation in this regard.

Signature

Student Name: Nikita Patle

Roll No.: CS-23411560

ACKNOWLEDGEMENT

I express my sincere gratitude to AICTE & EduSkills for providing me with the opportunity to undertake this online virtual internship. I would like to thank my industry mentor Mr./Ms. Sagarabala Behera for his/her valuable guidance, continuous support, and technical insights throughout the internship period.

I am also thankful to the Department of B.Tech (Computer Science and Engineering) for their encouragement and support.

Signature

Student Name: Nikita Patle
Roll No.: CS-23411560

TABLE OF CONTENTS

Chapter No.	Chapter Name	Page Nos.
	Cover Page	----
1.	Candidate's Declaration	i
2.	Acknowledgment	ii
3.	Internship Completion Certificate	----
4.	Project Description 4.1 Introduction 4.2 Organization Profile 4.3 Problem Statement 4.4 Project Objectives 4.5 Scope of the Project 4.6 Technologies and Tools Used 4.7 System Architecture 4.8 Methodology 4.9 Expected Outcomes 4.10 Certificates of Completion	1 1 1 1 2 2 2 2 6 6 8
5.	Bibliography/References	9

INTERNSHIP COMPLETION CERTIFICATE



PROJECT DESCRIPTION

4.1 Introduction

Industry-oriented learning through internships plays a crucial role in bridging the gap between academic knowledge and real-world applications. This online virtual internship provided hands-on exposure to Java Full Stack With Project and its application in modern software solutions.

Abstract

This project simulates essential components of a basic authentication system. It covers user registration with simple password hashing, validation of login attempts against stored credentials, security monitoring through login attempt tracking, and an account lockout mechanism. The project aims to demonstrate fundamental programming concepts in Java, including string manipulation, state management, conditional logic, and output formatting.

4.2 Organization Profile

Organization Name: EduSkills Academy

Domain: EdTech & Skill Development

Services: Virtual Internships

Vision: To empower students with industry-relevant skills.

Internship Overview

- Internship Mode: Online / Virtual
- Duration: 8 Weeks (Jun 2026 - Jul 2026)
- Platform Used: EduSkills LMS
- Role: Intern

4.3 Problem Statement

This project details the simulation of a basic authentication system, encompassing the critical functionalities of user registration, secure password handling, and login validation. It explores the implementation of a simple hashing algorithm for password security, alongside mechanisms for monitoring login attempts and enforcing account lockout policies. The system is designed to provide a foundational understanding of

authentication workflows, state management in processing, and the application of logical constructs in Java for building secure and responsive applications. The project serves as a practical exercise in developing core security features.

4.4 Project Objectives

- To design and implement a solution for the given problem statement.
- To utilize modern tools and algorithms.
- To optimize performance and code quality.

4.5 Scope of the Project

- Enhance password hashing using industry-standard algorithms like bcrypt or Argon2 for improved security.
- Implement more sophisticated security features such as rate limiting, CAPTCHA integration, or multi-factor authentication.
- Extend the system to support multiple user accounts and manage their states independently.

4.6 Technologies and Tools Used

Programming Language: Java

Concepts: String manipulation, Stream processing, State management, Conditional logic (if-else statements), Loops (for-loops), Data parsing, Input/Output operations, Algorithmic thinking.

4.7 System Architecture

The system is organised as five integrated modules, each implemented and tested as a separate problem.

Problem 1: User Registration and Simple Hashing

The task involves processing user registration data, provided in a comma-separated format (username,password). A 'simple hash' function must be implemented which transforms the provided password into a string consisting of its length followed by its first character. This processed hash is intended for storage and subsequent verification during login attempts.

```
import java.util.*;
public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
```

```
        List<String> results = new ArrayList<>();

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();

            if (line.trim().isEmpty()) {
                results.add("{\"key\":[]}");
            } else {
                // Parse 'user,pass'
                // Hash = pass length + first char
                // Format = 'user:hash'
                String[] parts = line.split(",");
                String username = parts[0];
                String password = parts[1];
                String hash = password.length() + "" + password.charAt(0);
                String transformed = username + ":" + hash;

                String escaped = transformed
                    .replace("\\", "\\\\")
                    .replace("\"", "\\\"");
                results.add("{\"key\":[\"" + escaped + "\"]}");
            }
        }
        scanner.close();

        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) System.out.print(",");
        }
        System.out.println("]");
    }
}
```

Problem 2: Login Validator Bug Fix

A critical bug within the login validation module prevents the correct comparison of attempted login credentials against stored user data. The existing code fails to accurately match the 'simple hash' of the entered password with the pre-calculated hash. The objective is to identify and rectify this logical error to ensure secure and accurate authentication.

```
import java.util.*;
public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();
            if (line.trim().isEmpty()) {
                results.add("{\"key\":[]}");
            } else {
                String[] p = line.split("\\|");
                String stored = p[0];
                String attempt = p[1];
                String attemptHash = attempt.length() + "" + attempt.charAt(0);

                // BUG: Using reference comparison instead of value
                String transformed = attemptHash.equals(stored) ? "SUCCESS" :
"DENIED";

                String escaped = transformed
                    .replace("\\", "\\\\")
                    .replace("\"", "\\\"");
                results.add("{\"key\":[\"" + escaped + "\"]}");
            }
        }
```

```
        scanner.close();
        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) System.out.print(",");
        }
        System.out.println("]");
    }
}
```

Problem 3: Tracking Login Attempts

To enhance security, the system needs to monitor consecutive failed login attempts. This task requires implementing a counter that increments with each 'FAIL' status and resets to zero upon a 'SUCCESS' status. For every input processed, the current count of consecutive failures must be outputted.

```
import java.util.*;
public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();
        int count = 0;
        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();
            if (line.trim().isEmpty()) {
                results.add("{\"key\":[]}");
            } else {
                if (line.equals("FAIL")) {
                    count++;
                } else if (line.equals("SUCCESS")) {
                    count = 0;
                }
                String transformed = String.valueOf(count);
                String escaped = transformed
                    .replace("\\", "\\\\")
                    .replace("\"", "\\\"");
                results.add("{\"key\":[\"" + escaped + "\"]}");
            }
        }
        scanner.close();
        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1)
                System.out.print(",");
        }
        System.out.println("]");
    }
}
```

Problem 4: Account Lockout Logic

Building upon the login attempt tracking, an account lockout policy must be implemented. The system should designate an account as 'LOCKED' if the consecutive failure counter reaches three or more. Otherwise, the account status remains 'ACTIVE'. The status ('ACTIVE' or 'LOCKED') for each login attempt needs to be reported.

```
import java.util.*;
public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();
        int count = 0;
        while (scanner.hasNextLine()) {
```

```
            String line = scanner.nextLine();
            if (line.trim().isEmpty()) {
                results.add("{\"key\":[]}");
            } else {
                if (line.equals("FAIL")) {
                    count++;
                } else if (line.equals("SUCCESS")) {
                    count = 0;
                }
                String transformed;
                if (count >= 3) {
                    transformed = "LOCKED";
                } else {
                    transformed = "ACTIVE";
                }
                String escaped = transformed
                    .replace("\\", "\\\\")
                    .replace("\"", "\\\"");
                results.add("{\"key\":[\"" + escaped + "\"]}");
            }
        }
        scanner.close();
        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) {
                System.out.print(",");
            }
        }
        System.out.println("]");
    }
}
```

Problem 5: Auth Audit Report

This task requires the generation of a formatted audit log from raw input data provided in CSV format, containing 'username,time,status'. The output should be presented in a structured format, specifically '[time] user: status', for each entry, facilitating a clear review of authentication events.

```
import java.util.*;
public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();
        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();
            if (line.trim().isEmpty()) {
                results.add("{\"key\":[]}");
            } else {
                String[] parts = line.split(",");
                String username = parts[0];
                String time = parts[1];
                String status = parts[2];
                String transformed = "[" + time + "] " + username + ": " + status;
                String escaped = transformed
                    .replace("\\", "\\\\")
                    .replace("\"", "\\\"");
                results.add("{\"key\":[\"" + escaped + "\"]}");
            }
        }
        scanner.close();
        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) {
                System.out.print(",");
            }
        }
        System.out.println("]");
    }
}
```

4.8 Methodology

The project was developed through a series of distinct, yet integrated, modules addressing specific aspects of authentication. Initially, user registration and a rudimentary password hashing technique were implemented. Subsequently, a login validation component was refined to correctly compare user-provided passwords with their hashed equivalents. Security monitoring was introduced by tracking consecutive login failures, with a mechanism to reset the failure count upon a successful login. This evolved into an account lockout policy, where repeated failures lead to account suspension. Finally, an audit reporting feature was added to generate a structured log of authentication events.

4.9 Expected Outcomes

Learning Outcomes

- Gained practical experience in implementing core authentication functionalities including registration, hashing, and login validation.
- Developed a deeper understanding of state management within stream processing scenarios.
- Enhanced proficiency in applying conditional logic and loops in Java for robust application development.
- Improved skills in parsing data, manipulating strings, and formatting output according to specified standards.

Challenges Faced & Solutions

Challenge: Implementing a robust and secure hashing mechanism with basic constraints.

Solution: Developed a simplified hashing algorithm that converts the password to its length followed by its first character, satisfying the project requirements for basic simulation.

Challenge: Accurately tracking and resetting consecutive login failure counts while managing account status.

Solution: Implemented a state variable to store the failure count, carefully designed conditional logic to increment on failure and reset on success, and integrated this with the account lockout policy logic.

Challenge: Parsing and formatting diverse input data for audit reporting.

Solution: Utilized Java's string manipulation capabilities to parse the CSV input and precisely format the output as required, ensuring clear and readable audit trails.

Conclusion

This project successfully simulates the fundamental aspects of a basic authentication system. By implementing user registration, simple hashing, login validation, attempt tracking, and account lockout, it provides a practical demonstration of core programming concepts and security principles. The successful completion of these tasks underscores the ability to manage application state, process data effectively, and build logical flows for essential system functionalities.

4.10 Certificates of Completion

TO WHOMSOEVER IT MAY CONCERN

This is to certify that Mr./Ms. Nikita Patle, a student of IILM University, Greater Noida, has successfully completed 8-Week (Jun 2026 - Jul 2026) AICTE – EduSkills Virtual Internship Program.

During the internship, he/she worked on the project titled "Basic Authentication Simulation". His/her performance during the internship was found to be satisfactory, and he/she demonstrated good technical and professional skills.

We wish him/her success in future endeavors.

Sagarabala Behera

Mentor, EduSkills Academy

Date: 11-08-2026

Certificate ID: 2026-D6B477DF2B (issued on Aug 11, 2026); AICTE Student ID: STU699737926b4921771517842; APAAR ID: 411239827479

BIBLIOGRAPHY / REFERENCES

[1] "Java Documentation." Oracle, Inc. (Relevant to Java language features and standard libraries).

[2] Bloch, Joshua. "Effective Java." Addison-Wesley Professional. (For best practices in Java development).

[3] Cormen, Leiserson, Rivest, and Stein. "Introduction to Algorithms." MIT Press. (For foundational algorithmic concepts).